TECHNICAL REPORT

Optical Communication — MATLAB/Simulink Modelling

Modelling and Simulation of Optical Transmitter Components

A technical report on the modelling and simulation of LED, Fabry-Pérot Laser, and Quantum-Well Laser using MATLAB/Simulink.

LED Model • Fabry-Pérot Laser • Quantum-Well Laser

AUTHOR

Puspendu Rana
Enrollment No.: 25560003
Department of Physics, IIT Roorkee
puspendu_r@ph.iitr.ac.in

SOURCE CODE

PuspenduPH/simulink-optical-transmitters

DATE

April 30, 2026

Table of Contents

1. Introduction **1**

2. Building a Simulink Model from Scratch **2**

2.1 Why Use Subsystems in Simulink . . . 2

2.2 Creating a Blank Model and Defining a Subsystem . . . 2

2.3 Creating and Editing a Mask . . . 3

2.4 Inserting Mask Parameters . . . 3

2.5 Writing the Mask Initialization Code . . . 3

2.6 Building the Internal Block Diagram . . . 4

2.7 Completing the Top-Level Model . . . 5

2.8 Configuring the Solver . . . 5

3. Modelling and Simulation of the Light-Emitting Diode **6**

3.1 Physics and Operating Principle . . . 6

3.2 Governing Equations . . . 6

3.3 Simulation Parameters . . . 6

3.4 Building the LED Subsystem . . . 7

3.5 Top-Level Model and Solver Configuration . . . 8

3.6 Simulink Block Diagrams . . . 8

3.7 Simulation Results and Analysis . . . 8

4. Modelling and Simulation of the Fabry-Pérot Laser Diode **9**

4.1 Physics and Operating Principle . . . 9

4.2 Governing Equations . . . 9

4.3 Simulation Parameters . . . 9

4.4 Building the FP Laser Subsystem . . . 10

4.5 Top-Level Model and Solver Configuration . . . 11

4.6 Simulink Block Diagrams . . . 11

4.7 Simulation Results and Analysis . . . 11

5. Modelling and Simulation of the Quantum-Well Laser Diode **12**

5.1 Physics and Operating Principle . . . 12

5.2 Governing Rate Equations . . . 12

5.3 Simulation Parameters . . . 12

5.4 Building the QW Laser Subsystem . . . 12

5.5 Top-Level Model and Test Harness . . . 13

5.6 Solver Configuration . . . 13

5.7 Simulink Block Diagrams . . . 14

5.8 Simulation Results and Analysis . . . 15

6. Summary and Conclusions **15**

List of Figures

1 Simulink block diagrams depicting the complete test harness along with the inner transfer-function calculation. 8

2 Simulated optical power output (mW) demonstrating the LED's transient response limit. 8

3 Simulink designs tracking external pulse modulation and internal transfer routing for the FP Laser. . 11

4 Simulated power spectrum captures identifying non-ideal laser ringing against rapid voltage switches. . 11

5 Simulink layouts for the QW Laser displaying the Hot-Start vs Cold-Start test benches, alongside the complex time-domain integration core. 14

6 Output profiles capturing turn-on delay (a), instantaneous clamped gain (b), and the realistic progression from empty cavity to steady-state modulation (c). 15

List of Tables

1 Comparison of flat model design versus subsystem-based modular design. 2

2 Fields to configure for each mask parameter entry. 3

3 Recommended solver settings for each optical transmitter component. 6

4 Simulation parameters for the LED model. 7

5 Simulation parameters defining the multimode Fabry-Pérot Laser. 10

6 Simulation parameters for the Quantum-Well LASER model. 12

7 Comparative summary of the three modeled optical transmitters. 16

Abstract

This report presents the modelling and simulation of three fundamental optical transmitter components, a Light-Emitting Diode (LED), a Fabry-Pérot (FP) Laser Diode, and a Quantum-Well (QW) Laser Diode using the MATLAB/Simulink environment. The governing equations and physical parameters are taken from the reference literature [1], which provides a comprehensive Simulink toolbox framework for simulation and analysis of optical fiber communication links.

For the LED and FP Laser, we used a linear small-signal frequency-domain approach, that represents each device as a cascade of a static DC gain and a continuous-time transfer function in the Laplace domain. The LED is characterised by a first-order low-pass response governed by the radiative carrier lifetime, while the FP Laser exhibits a second-order resonant characteristic that gives rise to relaxation oscillations. For the QW Laser, the small-signal approximation is insufficient; so, we directly implement the coupled nonlinear carrier and photon density rate equations using closed-loop integrator feedback loops in Simulink.

All three models are implemented as reusable, masked Simulink subsystems with custom MATLAB class-based initialization scripts that handle unit conversion and the computation of derived gain and transfer function coefficients prior to the simulation. This report provides a step-by-step guide for building such masked subsystems from scratch, followed by detailed discussions of the model architecture, solver configuration, and scope-based simulation results for each device. The simulation results are analyzed in the context of the underlying semiconductor physics, highlighting phenomena such as the LED bandwidth ceiling, laser relaxation oscillations, carrier turn-on delay, and gain clamping under pre-biased operation.

1. Introduction

Optical fiber communication systems are the backbone of modern high-speed data networks. Their applications range from metropolitan-area links to transoceanic submarine cables. At the transmitting end of every such link there is an electro-optic transducer, typically a semiconductor Light-Emitting Diode or a Laser Diode, whose task is to convert a time-varying electrical current into a modulated optical power signal. The dynamic characteristics of this transducer directly limit the achievable modulation bandwidth and signal quality of the entire communication system. Accurate modelling of these devices is therefore crucial both for understanding their physical limitations and for designing practical fiber-optic links.

MATLAB/Simulink is a widely used environment for block-diagram-based simulation of dynamic systems. Its rich library of continuous-time blocks like Gain, Transfer Function, Integrator, Sum, and Product makes it particularly well-suited for translating physics-derived equations directly into executable simulation models without writing explicit differential equation solvers. The use of *masked subsystems* further enhances modularity: each device is encapsulated behind a single named block with a user-friendly parameter dialog, that allows the model to be reused across different simulation scenarios by simply changing the dialog inputs.

This report utilizes the modelling framework presented in [1] as its theoretical foundation. The governing equations, physical parameters, and model architectures follow the same methodology described in the paper. We implemented and simulated three optical transmitter components of increasing complexity:

1. **Light-Emitting Diode (LED):** Modelled as a first-order linear system driven by a small-signal frequency-domain transfer function. The key dynamic is the exponential transient characteristics governed by the radiative carrier lifetime τ_r.
2. **Fabry-Pérot (FP) Laser Diode:** Modelled as a second-order resonant system whose dynamic transfer function captures the relaxation oscillations arising from the coupling between the carrier and photon populations inside the laser cavity.
3. **Quantum-Well (QW) Laser Diode:** Modelled using the full nonlinear coupled rate equations for carrier density $N(t)$ and photon density $S(t)$, implemented via closed-loop Simulink integrators. This large-signal model captures phenomena such as carrier turn-on delay and gain clamping that are invisible to the linearised transfer-function approach.

The remainder of this report is organized as follows. Section 2 provides a beginner-oriented, step-by-step tutorial on building a reusable masked Simulink subsystem from scratch covering subsystem creation, mask editing, parameter dialog design, initialization code, internal block wiring, and solver tuning. Sections 3, 4, and 5 present the modelling and simulation of the LED, FP Laser, and QW Laser, respectively, each covering the governing equations, physical parameters, subsystem architecture, top-level model configuration, and a physics-based interpretation of the simulation results. Section 6 summarises the key findings across all three models and discusses the physical insights gained.

2. Building a Simulink Model from Scratch

This section provides a self-contained, step-by-step guide for building a reusable, masked Simulink component from a blank canvas. The workflow discussed here is the exact procedure we follow when building each of the three optical transmitter models in Sections 3, 4, and 5. Readers who are already familiar with Simulink subsystem masking may skip and proceed to the device-specific sections.

2.1 Why Use Subsystems in Simulink

When building a Simulink model of a physical device for example, an LED or a laser diode a simple approach would be placing every individual math block (multipliers, adders, transfer functions) directly on the top-level canvas. This quickly becomes unmanageable as the model grows in complexity. Table 1 summarises the key problems with a flat model and the corresponding solutions provided by a subsystem-based design.

Problem with a flat model	Solution provided by a subsystem
Canvas becomes a spaghetti diagram	All internals are hidden behind a single, named block icon
Repeating the same calculation in multiple models	The subsystem file is reused; change once, update everywhere
Each parameter must be set by editing deep internal blocks	The **Mask** exposes a single clean dialog box for all inputs
Static pre-calculations (efficiencies, unit conversions) waste computation inside the diagram	The **Mask Initialization Script** runs once at compile time
Collaborators must understand every internal block	They only see the labelled ports and the dialog box

Table 1: Comparison of flat model design versus subsystem-based modular design.

The core principle is straightforward: a subsystem groups related blocks into one reusable component. A *mask* then wraps that subsystem with a custom dialog box and a background initialization script, reducing the visual model to only the blocks that absolutely must run at every simulation time step.

2.2 Creating a Blank Model and Defining a Subsystem

A subsystem should represent exactly one physical component or one well-defined mathematical stage (e.g., *optical transmitter*, *fiber channel*, *photodetector*). In general, if a group of blocks always appears together, can be characterized by a fixed set of physical parameters, and the rest of the model can be simplified by using only input/output ports, then it should be extracted into a subsystem.

Method A – Empty subsystem (recommended for new components).

1. In the Simulink toolstrip, navigate to **File** → **New** → **Subsystem**. Alternatively, open a Blank Model and drag a `Subsystem` block from the *Ports & Subsystems* library onto the canvas.
2. Double-click the block to open its internal canvas.
3. Simulink automatically places one **Inport** (`In1`) and one **Outport** (`Out1`) to define the signal boundary.

Method B – Convert existing blocks.

1. On the main canvas, drag a selection rectangle around the blocks to be grouped.
2. Right-click the selection → **Create Subsystem from Selection**.
3. Simulink automatically creates the Inports and Outports at the boundary.

After creating the subsystem, rename the ports to physically meaningful labels. Double-click each **Inport** block and change its name to, e.g., `I_in` (drive current in Amperes); do the same for the **Outport**, e.g., `P_out` (optical power in Watts). These labels will appear on the block icon in the parent model.

2.3 Creating and Editing a Mask

A **mask** is a layer placed on top of a subsystem that provides three capabilities [2]:

- A custom **dialog box** (parameter prompts instead of raw block properties).
- A background **initialization script** (MATLAB code that runs before simulation).
- An optional custom **icon** and **description**.

Opening the Mask Editor. With the subsystem block selected (or its canvas open), use any of the following methods:

- **Toolstrip:** Select the **Subsystem** tab → click **Edit Mask**.
- **Right-click:** Right-click the subsystem block on the parent canvas → **Mask** → **Edit Mask...**
- **Keyboard shortcut:** `Ctrl+M` while the subsystem block is selected.

The Mask Editor window contains four tabs:

1. **Icon & Ports** – custom drawing commands for the block icon.
2. **Parameters & Dialog** – define the user input fields.
3. **Initialization** – MATLAB code that runs at model initialisation.
4. **Documentation** – help text and description.

2.4 Inserting Mask Parameters

Every physical quantity the user must supply is defined in the **Parameters & Dialog** tab of the Mask Editor. For each parameter, the fields listed in table 2 must be configured.

Field	Purpose	Example
Name	MATLAB variable name used in the init script. Must be a valid identifier.	`lambda`
Prompt	Human-readable label the user sees in the dialog box.	`Emission Wavelength (nm)`
Type	Widget type: `edit` (text box), `checkbox`, `popup`, etc.	`edit`
Evaluate	If checked, Simulink evaluates the entered string as a MATLAB expression.	Checked
Tunable	Whether the value can change during simulation.	Typically unchecked

Table 2: Fields to configure for each mask parameter entry.

Naming convention. We keep the **Name** field identical to the symbol used in the mathematical model (e.g., `tau_r`, `R1`, `gamma`). This makes the initialization script self-documenting and minimises the chance of transcription errors.

Organising parameters into groups. For models with many parameters, we insert **Group Box** separators between logically related parameters. In the Mask Editor, click the small dropdown arrow on the + button → select **Group Box**, then drag parameters into the group. For example, a laser diode mask might use three groups: *Cavity & Structural Parameters*, *Time Constants & Lifetimes*, and *Operating Currents*. After configuring all parameters, click **Apply** then **OK**. Double-clicking the subsystem block in the parent model should now display the custom dialog box rather than the raw internal canvas.

2.5 Writing the Mask Initialization Code

The initialization code is the most critical step in the subsystem workflow. It is a MATLAB script that Simulink executes **once** when the model is compiled (or whenever mask parameters change). Its responsibilities are:

1. **Read** the user-supplied values from the mask dialog.

2. **Convert units** to SI if the dialog accepts practical units (nm, mA, ns).
3. **Calculate** all derived quantities (efficiencies, gain, transfer function coefficients).
4. **Write** the computed results back to the mask workspace so that blocks inside the subsystem can reference them by name.

Mandatory boilerplate structure. All initialization scripts in this project follow a rigid `classdef`-based structure. The code must be wrapped in a MATLAB class with a `Static` method named `MaskInitialization` that accepts `maskInitContext` as its single argument. Deviating from this structure will cause Simulink to fail silently or skip the script entirely. The canonical template is shown in listing 1.

```
classdef <ClassName>          % e.g., laser_tf_init, led_mask_init
    methods(Static)
        function MaskInitialization(maskInitContext)
            % Access the mask workspace object
            ws = maskInitContext.MaskWorkspace;

            % Section 1: Retrieve parameters from dialog
            <param> = ws.get('<param>');

            % Section 2: Define universal physical constants
            h = 6.62e-34;   % Planck's constant (J.s)
            c = 2.9979e8;   % Speed of light (m/s)
            q = 1.602e-19;  % Elementary charge (C)

            % Section 3: Convert units to SI (if needed)
            % e.g., lambda_m = lambda * 1e-9;

            % Section 4: Compute derived quantities
            % e.g., HT_0 = (h*c/(lambda_m*q)) * eta_int * eta_ext;

            % Section 5: Push results back to workspace
            ws.set('<result_var>', <result_var>);
        end
    end
end
```

Listing 1: Mandatory boilerplate structure for mask initialization scripts.

Why a classdef? Simulink's mask initialization engine requires the code to be a proper MATLAB class with a static method. This convention isolates the initialization code from the base workspace, allows Simulink to call it safely during model compilation, and prevents naming conflicts between multiple masked subsystems.

Critical pitfall. Variables computed inside the initialization script do **not** automatically appear in the mask workspace for internal blocks to use. Each result variable must be explicitly pushed back via `ws.set()`. Forgetting this call is the single most common bug; the block diagram will error with "undefined variable" even though the script ran correctly.

2.6 Building the Internal Block Diagram

With the mask configured and the initialization script ready to supply computed variables, we now build the actual signal-processing network inside the subsystem canvas.

Design principle: keep the diagram minimal. The initialization script has already handled every static calculation. The blocks remaining in the diagram should only represent operations that *must* be evaluated at every simulation time step:

- Multiplication by a computed gain constant → **Gain** block.
- Dynamic filtering / frequency response → **Transfer Fcn** block [3].
- Signal routing → **Sum** block, **Mux/Demux**.

For a transfer-function-based component (such as the LED or FP Laser), the canonical internal signal chain reduces to just two functional blocks:

$$\texttt{[Inport]} \longrightarrow \texttt{[Gain:}\ \ H_T(0)\texttt{]} \longrightarrow \texttt{[Transfer Fcn:}\ \ H_T^*(s)\texttt{]} \longrightarrow \texttt{[Outport]} \tag{1}$$

Configuring the Gain block. Drag a **Gain** block from the *Math Operations* library onto the subsystem canvas. Double-click it and set the **Gain** field to the variable name computed by the initialization script, e.g., `HT_0`. Simulink resolves this name from the mask workspace at simulation start.

Configuring the Transfer Fcn block. Drag a **Transfer Fcn** block from the *Continuous* library. Double-click and set:

- **Numerator coefficients:** a MATLAB vector expression, e.g., `[f0_sq]`.
- **Denominator coefficients:** e.g., `[1, beta, f0_sq]`.

Simulink expects coefficients in descending power order. For example, the denominator $s^2 + \beta s + f_0^2$ is entered as `[1, beta, f0_sq]`, corresponding to $1 \cdot s^2 + \beta \cdot s^1 + f_0^2 \cdot s^0$.

Wiring the blocks. Hover over an output port until the crosshair cursor appears, then click-drag to the input port of the next block. Alternatively, click the source block, then `Ctrl+click` the destination block and Simulink will auto-route the wire. Signal lines can be labelled by double-clicking them (e.g., label the line between the Gain and Transfer Fcn blocks as `P_base`).

2.7 Completing the Top-Level Model

Once the subsystem is fully self-contained, we place it on the main model canvas and connect it to the stimulus source and measurement sink.

Adding the subsystem. The subsystem block can be added to the main canvas by one of three methods:

1. **Drag-and-drop from file:** Locate the standalone `.slx` file in the MATLAB file browser and drag it onto the canvas.
2. **Copy-paste:** Open the source model, `Ctrl+C` the subsystem block, switch to the target canvas, and `Ctrl+V`.
3. **Library Browser:** `Ctrl+Shift+L` → navigate to your custom library → drag the block onto the canvas.

Connecting the drive signal. The subsystem block displays its named ports (`I_in`, `P_out`, etc.) as small arrows. Draw signal lines from the source blocks (e.g., a Pulse Generator, or a Sum block combining a constant bias with a pulse) to the input port. Similarly, connect the output port to a Scope or other measurement sink.

Entering parameters. Double-click the subsystem block on the main canvas, the custom mask dialog box appears. Fill in all the physical parameters and click **OK**. Simulink will call the initialization script and populate the mask workspace before the simulation starts.

2.8 Configuring the Solver

The solver must be configured tightly enough to capture the fastest dynamics in the model. Open the solver settings via **Modeling** tab → **Model Settings** (or `Ctrl+E`) [4].

The most critical setting is the **maximum step size**. If the step size is too large, the solver will skip over fast transients (e.g., relaxation oscillations in a laser), producing noisy plots or missing the peak ringing entirely. In general, the maximum step size should be set to at least 10× smaller than the fastest time constant in the model. Table 3 provides recommended settings for the devices studied in this report.

Device	Critical time constant	Max step size	Solver
LED	Radiative lifetime τ_r (ns range)	`1e-10` s (0.1 ns)	Variable-step (`ode45`)
FP Laser	Photon lifetime τ_{ph} (ps range)	`1e-12` s (1 ps)	Fixed-step (`ode4`)
QW Laser	Photon lifetime τ_p (ps range)	`1e-13` s (100 fs)	Fixed-step (`ode4`)

Table 3: Recommended solver settings for each optical transmitter component.

The simulation stop time should be set to approximately 10× the longest period of interest. For example, if the pulse generator operates at 500 MHz (period = 2 ns), a stop time of `10e-9` (10 ns) provides five complete modulation cycles for analysis.

3. Modelling and Simulation of the Light-Emitting Diode

3.1 Physics and Operating Principle

The behavior of a Light-Emitting Diode (LED) in an optical fiber link can be modelled using a small-signal frequency-domain approach. In this linear regime, the output optical power $P_e(f)$ (in Watts) is directly proportional to the injected drive current $I_d(f)$ (in Amperes). The dynamic relationship is characterized by the LED's total transfer function $H_T(f)$:

$$P_e(f) = H_T(f) \cdot I_d(f) \tag{2}$$

The total transfer function is divided into two distinct components: a static DC gain $H_T(0)$ and a dynamic normalized frequency response $H_T^*(f)$. This mathematical separation allows the Simulink model to segregate steady-state efficiency calculations from transient dynamics.

3.2 Governing Equations

The static part of the transfer function, $H_T(0)$, represents the overall steady-state quantum efficiency of the LED. It dictates the ratio of generated optical power to injected electrical current. It is given by:

$$H_T(0) = \left(\frac{hc}{\lambda q}\right) \eta_{int}\eta_{inj}\eta_{ext} \tag{3}$$

where λ is the emission wavelength, h is Planck's constant, c is the speed of light, and q is the elementary charge. The component efficiencies are the internal quantum efficiency η_{int} and external quantum efficiency η_{ext}, calculated as:

$$\eta_{int} = \frac{\tau_{nr}}{\tau_{nr} + \tau_r}, \qquad \eta_{ext} = \left[1 - \left(\frac{n_s - n_a}{n_s + n_a}\right)^2\right]\left[1 - \cos\left(\frac{n_a}{n_s}\right)\right] \tag{4}$$

where τ_{nr} and τ_r are the non-radiative and radiative carrier lifetimes, respectively.

The dynamic part, $H_T^*(f)$, models the transient response caused by the carrier recombination lifetime. The response features a first-order low-pass characteristic:

$$H_T^*(f) = \frac{1}{1 + jf/f_c} \quad \text{where} \quad f_c = \frac{1}{2\pi\tau_r} \tag{5}$$

By making the Laplace substitution $s = j2\pi f$, this equation translates perfectly into a continuous-time first-order low-pass filter:

$$H_T^*(s) = \frac{1}{\tau_r s + 1} \tag{6}$$

3.3 Simulation Parameters

The physical constants and operating parameters required to simulate the LED are summarised in table 4.

Parameter	Symbol	Description	Unit	Value
Emission wavelength	λ	Operating wavelength	m	850×10^{-9}
Radiative lifetime	τ_r	Carrier radiative lifetime	s	5.0×10^{-9}
Non-radiative lifetime	τ_{nr}	Carrier non-radiative lifetime	s	50.0×10^{-9}
Injection efficiency	η_{inj}	Current-injection efficiency	-	0.90
Semiconductor index	n_s	Refractive index of semiconductor	-	3.60
Media index	n_a	Refractive index of surrounding media	-	1.00

Table 4: Simulation parameters for the LED model.

3.4 Building the LED Subsystem

To optimize computational efficiency and keep the visual block diagram clean, static calculations are moved to an initialization script within the subsystem mask. The inputs for wavelength and lifetimes are requested in standard SI units (meters, seconds), avoiding runtime unit conversions. The complete mask initialization MATLAB code is provided in listing 2.

```
% Input Argument of MaskInitialization function is a struct with properties
% BlockHandle, MaskObject and MaskWorkspace. Example: maskInitContext.BlockHandle

% Input Argument of Parameter Callbacks is a struct with properties
% BlockHandle and ParameterObject. Example: callbackContext.ParameterObject

classdef led_mask_init
    methods(Static)
        function MaskInitialization(maskInitContext)
            % Access the mask workspace object
            ws = maskInitContext.MaskWorkspace;

            % 1. Retrieve parameters from the dialog box
            lambda_ns  = ws.get('lambda');
            tau_r_ns   = ws.get('tau_r');
            tau_nr_ns  = ws.get('tau_nr');
            eta_inj = ws.get('eta_inj');
            n_s     = ws.get('n_s');
            n_a     = ws.get('n_a');

            % Universal Constants
            h = 6.62e-34;        % plank's constant
            c = 2.9979e8;        % Speed of Light
            q = 1.602e-19;       % Electron's Charge

            % converting variables back to SI units
            lambda = lambda_ns*1e-9;
            tau_r = tau_r_ns*1e-9;
            tau_nr = tau_nr_ns*1e-9;

            % 2. Calculating Efficiencies
            eta_int = tau_nr / (tau_nr + tau_r);
            eta_ext = (1 - ((n_s - n_a)/(n_s + n_a))^2) * (1 - cos(n_a/n_s));

            % 3. Calculating HT_0
            HT_0 = ((h * c) / (lambda * q)) * eta_int * eta_inj * eta_ext;

            % 4. Push the calculated variables back to the mask workspace
            ws.set('HT_0', HT_0);
            ws.set('tau_r', tau_r);
        end
    end
end
```

Listing 2: Mask initialization code for the LED subsystem evaluating static properties.

With the mathematical preprocessing managed by the script, the internals of the subsystem contain just two blocks in series: a **Gain** block set to `HT_0` for scaling input current to baseline optical power, and a **Transfer Fcn** block with denominator `[tau_r 1]` to implement the continuous-time dynamics of eq. (6).

3.5 Top-Level Model and Solver Configuration

The complete top-level integration of the LED requires an appropriate electrical stimulus. Unlike standard symmetric AC models, an optical diode operates uniquely with a unipolar forward current. The stimulus is provided by a Pulse Generator configured for a 25 MHz square wave swinging between 0 and 50 mA. The external model then uses a generic optical power sink (in this test bench, simply a multiplier translating Watts to milliwatts) and an oscilloscope block to observe the output. We have also used gain blocks to level up the output power of the LED to match the scale of the input current for better visualization on the scope. We utilized the **Mux** block to combine multiple scaler signals like the input current and output power into a single vector signal for simultaneous display on the scope. The same thing can be achieved by using multiple ports in the scope block and connecting the signals directly to each of the ports. This is a matter of preference and does not affect the simulation results. The gain block are used to display the output power and the input current on the same plot. However, we can directly visualize the input current and output power in two different subplots by choosing a 2x1 or 1x2 layout in the scope block. This way, we can avoid using gain blocks and directly compare the input current and output power without scaling.

To properly measure the fast exponential characteristics, the Simulink solver's **Max step size** must be strictly bound to `1e-10` seconds (0.1 ns). This prevents a variable-step ODE solver from interpolating over the rise/fall curves dictated by the nanosecond-scale radiative lifetime τ_r.

3.6 Simulink Block Diagrams

The visual implementation of the test bench and the encapsulated subsystem are presented in fig. 1.

(a) Top-level model of the LED test circuit.

(b) Internal block diagram of the LED subsystem.

Figure 1: Simulink block diagrams depicting the complete test harness along with the inner transfer-function calculation.

3.7 Simulation Results and Analysis

The response of the LED subsystem to the 50 mA current pulses is depicted in the scope trace shown in fig. 2.

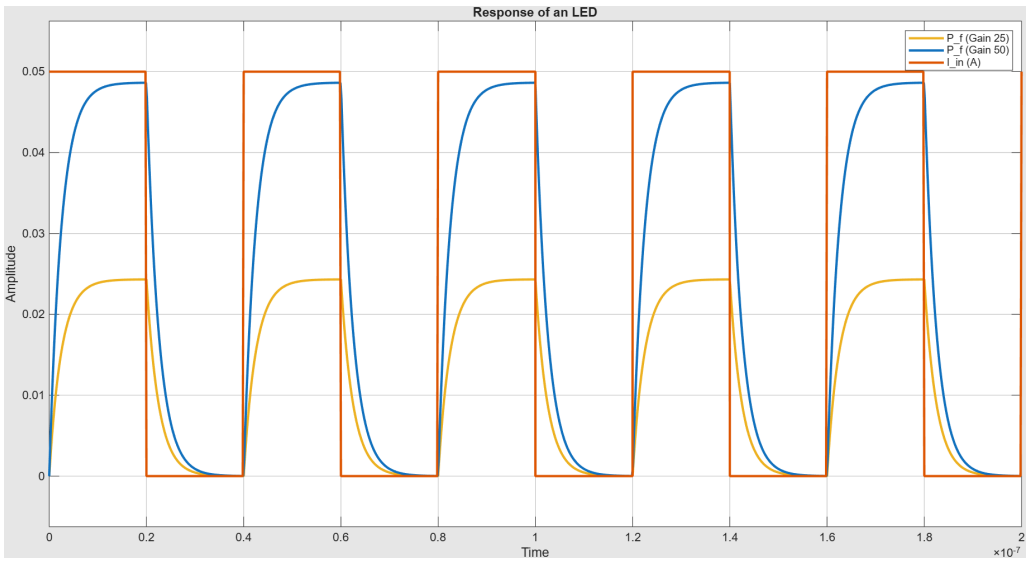


Figure 2: Simulated optical power output (mW) demonstrating the LED's transient response limit.

Observation: When the drive current steps abruptly to 50 mA, the optical power exhibits a distinct exponential rise rather than an instantaneous rectangular jump. On the other hand, at the trailing edge of the drive pulse, the power decays exponentially back to zero at an identical rate.

Physics: The rate of this exponential profile is governed entirely by the radiative carrier lifetime τ_r. This behavior is the direct result in the time domain of the first-order low-pass filter modeled at $f_c = 1/(2\pi\tau_r)$. Higher-frequency temporal components of the 25 MHz square-wave stimulus are progressively attenuated. Additionally, since the governing transfer function is linear, the time constants determining both the carrier accumulation (rise) and recombination (fall) are symmetric.

Implication: The finite optical response limits the 3 dB modulation bandwidth of the device to $f_{3\text{dB}} \approx 1/(2\pi\tau_r)$. Since radiative recombination times are fundamentally bounded to the nanosecond range, practical LED bandwidths are ceiling-limited to tens or low hundreds of megahertz. This inherent restriction explains why LEDs are predominantly used in moderate-speed optical data links, while high-performance networks strictly deploy laser diodes.

4. Modelling and Simulation of the Fabry-Pérot Laser Diode

4.1 Physics and Operating Principle

The behavior of a multimode Fabry-Pérot (FP) laser diode can be effectively analyzed using a small-signal frequency-domain model. When driven by an injected current $I_d(f)$, the output optical power $P_e(f)$ (in Watts) is determined by the total transfer function $H_T(f)$ of the laser:

$$P_e(f) = H_T(f) \cdot I_d(f) \tag{7}$$

Much like the LED, the total transfer function $H_T(f)$ is decomposed into a static DC gain $H_T(0)$ and a dynamic frequency response $H_T^*(f)$:

$$H_T(f) = H_T(0) \cdot H_T^*(f) \tag{8}$$

4.2 Governing Equations

The static quantum efficiency $H_T(0)$ evaluates the performance of the light source (in Watts/Ampere) when operated securely above the threshold current ($I_d > I_{th}$). It is determined by structural factors and the differential current ratio:

$$H_T(0) = \left(\frac{hc}{\lambda q}\right) \eta_{int}\eta_{ext} \left[\frac{I_d - I_{th}}{I_d}\right] \tag{9}$$

where η_{int} and η_{ext} are the internal and external quantum efficiencies. They are defined by internal cavity losses γ, facet reflectivity R_1, cavity length L, and the non-radiative τ_{nr} and radiative τ_r carrier lifetimes:

$$\eta_{int} = \frac{\tau_{nr}}{\tau_{nr} + \tau_r}, \qquad \eta_{ext} = \frac{\ln(1/R_1)}{\gamma L + \ln(1/R_1)} \tag{10}$$

The dynamic portion, $H_T^*(s)$, dictates the high-frequency transient response, specifically the relaxation oscillations induced by rapid current changes. Using the Laplace formulation ($s = j2\pi f$), the frequency-plane physics translates into a continuous-time second-order resonant transfer function:

$$H_T^*(s) = \frac{f_0^2}{s^2 + \beta s + f_0^2} \tag{11}$$

The resonant frequency squared (f_0^2) and damping factor (β) rely inherently on the interplay between carrier lifetime τ_{sp} and photon lifetime τ_{ph}, as well as the initial pre-bias current I_0:

$$f_0^2 = \frac{I_0 - I_{th}}{\tau_{sp}\tau_{ph}I_{th}}, \qquad \beta = \frac{I_0}{\tau_{sp}I_{th}} \tag{12}$$

4.3 Simulation Parameters

The physical dimensions, operating currents, and optical constraints characterising the specific FP Laser modelled are given in table 5. All of these values are sourced from the base paper [1].

Parameter	Symbol	Description	Unit	Value
Emission wavelength	λ	Operating wavelength of the laser	nm	1550
Non-radiative lifetime	τ_{nr}	Carrier non-radiative lifetime	ns	5
Radiative lifetime	τ_r	Carrier radiative lifetime	ns	0.5
Cavity dimension	L	Longitudinal length of the cavity	m	3×10^{-4}
Loss coefficient	γ	Internal optical losses	m^{-1}	35×10^{-4}
Mirror reflectancy	R_1	Reflectivity of the cavity facets	-	0.32
Carrier lifetime	τ_{sp}	Spontaneous emission carrier lifetime	ns	0.02
Photon lifetime	τ_{ph}	Cavity photon lifetime	ns	0.001
Injection current	I_d	Driving injection current	mA	2
Threshold current	I_{th}	Lasing threshold current	mA	0.5
Pre-bias current	I_0	Constant pre-bias current	mA	1

Table 5: Simulation parameters defining the multimode Fabry-Pérot Laser.

4.4 Building the FP Laser Subsystem

Due to the complex interplay of structural constants and lifetimes, the static baseline multiplier $H_T(0)$ and damping terms (f_0^2, β) are pushed completely to the subsystem's background mask script. The script automatically converts the inputs from practical units (nm, mA, ns) into SI baselines before configuring the visual blocks.

```
classdef laser_tf_init
    methods(Static)
        function MaskInitialization(maskInitContext)
            ws = maskInitContext.MaskWorkspace;

            % Retrieve parameters
            lambda = ws.get('lambda');  tau_nr = ws.get('tau_nr');
            tau_r  = ws.get('tau_r');   L      = ws.get('L');
            gamma  = ws.get('gamma');   R1     = ws.get('R1');
            I_d    = ws.get('I_d');     I_th   = ws.get('I_th');
            I_0    = ws.get('I_0');     tau_sp = ws.get('tau_sp');
            tau_ph = ws.get('tau_ph');

            % Format SI units
            lambda_m = lambda * 1e-9;
            I_d_A = I_d*1e-3; I_0_A = I_0*1e-3; I_th_A = I_th*1e-3;
            tau_sp_s = tau_sp * 1e-9; tau_ph_s = tau_ph * 1e-9;
            h = 6.62e-34; c = 2.9979e8; q = 1.602e-19;

            % Calculate Efficiencies & DC Transfer Function
            eta_int = tau_nr / (tau_nr + tau_r);
            eta_ext = log(1/R1) / (gamma * L + log(1/R1));
            mod_ratio = (I_d_A - I_th_A) / I_d_A;
            HT_0 = ((h * c) / (lambda_m * q)) * eta_int * eta_ext * mod_ratio;

            % Calculate Frequency Resonance Poles
            f0_sq = (I_0_A - I_th_A) / (tau_sp_s * tau_ph_s * I_th_A);
            beta  = I_0_A / (tau_sp_s * I_th_A);

            % Push to Workspace
            ws.set('f0_sq', f0_sq);
            ws.set('beta', beta);
            ws.set('HT_0', HT_0);
        end
    end
end
```

Listing 3: Mask initialization script for the FP Laser calculating resonant damping constraints.

Because the initialization script resolves the heavy lifting externally, the internal view of the LASER relies strictly on processing block primitives. The raw current I_{in} scales through a **Gain** block assigned the value `HT_0`. It then strikes a 2nd-order **Transfer Fcn** block defining the numerator as `[f0_sq]` and the denominator as `[1, beta, f0_sq]`.

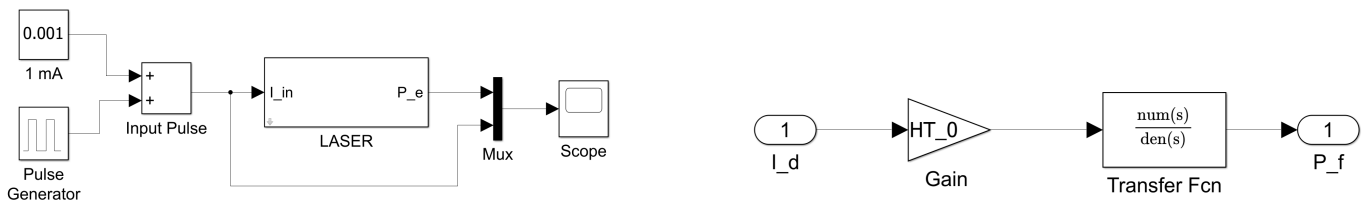
4.5 Top-Level Model and Solver Configuration

The small-signal approximation is valid as long as the drive current does not drop below the threshold current I_{th}. If the input current were to fall below I_{th}, the laser would experience a significant turn-on delay due to the need to build up carrier density before lasing can occur. This would require a more complex nonlinear rate-equation model to accurately capture the dynamics, which is beyond the scope of this small-signal transfer function approach. Since this is an uncoupled small-signal model, the drive test-bench can be configured with a simple DC bias current set to 1 mA (just above the 0.5 mA threshold) to ensure continuous lasing operation.

A 2 mA amplitude Pulse Generator running a 500 MHz pulse width overlaps this bias, driving the core signal through a generic Sum block. Crucially, the 1 ps photon lifetime τ_{ph} triggers wildly fast relaxation transients. The Simulink scope would fragment or completely bypass rendering these waves if standard variable-step interpolation applies. Thus, a fixed-step Runge-Kutta ODE4 solver clamped precisely to a 1×10^{-12} s (1 ps) tick rate operates at standard testing lengths.

4.6 Simulink Block Diagrams

Visual confirmation of the test circuit framework is shown side-by-side in fig. 3.



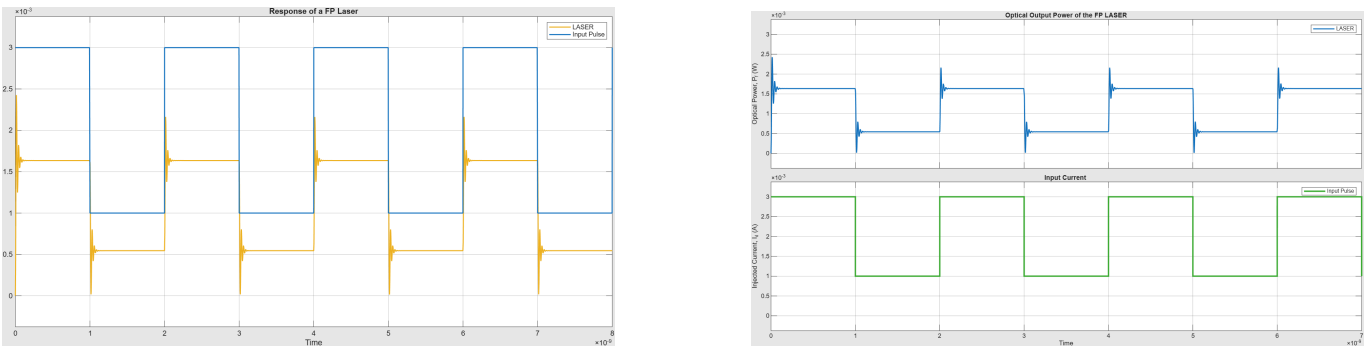
(a) Top-level model of the FP Laser pulse drive.

(b) Internal block schematic isolated scaling transfer.

Figure 3: Simulink designs tracking external pulse modulation and internal transfer routing for the FP Laser.

4.7 Simulation Results and Analysis

Exposing the diode input to a 500 MHz transition results in prominent optical artifacts documented in fig. 4.



(a) Power vs Time single-panel scope plot.

(b) Dual-panel trace for current and power steps.

Figure 4: Simulated power spectrum captures identifying non-ideal laser ringing against rapid voltage switches.

Observation: Upon each vertical rising step from 1 mA to 3 mA in the input source, the resulting optical profile overshoots the targeted stability ceiling. This induces a sharp high-frequency "ringing" dynamic that decays incrementally before flattening into a stable high-power emission phase.

Physics: The ringing behavior is due to the 2nd-order resonant pole present in the dynamic transfer function $H_T^*(s)$. This phenomenon is known as relaxation oscillations. When a large amount of of electrons are injected into the cavity, the carrier density N builds up rapidly while the photon density S initially lags behind. The carriers suddenly trigger a burst of stimulated emission, releasing a large number of photons and depleting the carrier population. The laser then falls below the threshold condition until the carriers can build up again, creating an oscillatory exchange between N and S that is heavily influenced by the resonant frequency f_0.

Implication: Biasing with a greater baseline current I_0 above the threshold I_{th} increases the damping factor β, which can help reduce the amplitude of relaxation oscillations. However, this also increases the resonant frequency f_0, which

may lead to faster oscillations that are more difficult to filter out. Therefore, there is a trade-off between reducing oscillation amplitude and increasing oscillation frequency when adjusting the bias current. Relaxation oscillations can be mitigated by optimizing the laser design to achieve a higher photon lifetime τ_{ph} and a lower spontaneous emission lifetime τ_{sp}, which can help to increase the damping of the oscillations.

5. Modelling and Simulation of the Quantum-Well Laser Diode

5.1 Physics and Operating Principle

Unlike standard LEDs and linearized Fabry-Pérot lasers which can be modeled using small-signal frequency-domain transfer functions, accurately capturing the large-signal transient dynamics of a Quantum-Well (QW) LASER requires solving its non-linear rate equations in the time domain.

The Simulink model directly implements the coupled differential equations governing the carrier density $N(t)$ and photon density $S(t)$ within the active region of the laser cavity.

5.2 Governing Rate Equations

The carrier density rate equation dictating the electron populations is defined as:

$$\frac{dN}{dt} = \frac{I}{qV_{act}} - g_0(N - N_0)(1 - \epsilon S)S - \frac{N}{\tau_n} + \frac{N_e}{\tau_n} \tag{13}$$

The coupled photon density rate equation dictating the internal optical mode is defined as:

$$\frac{dS}{dt} = \Gamma g_0(N - N_0)(1 - \epsilon S)S + \frac{\Gamma \beta N}{\tau_n} - \frac{S}{\tau_p} \tag{14}$$

The emitted optical output power $P_f(t)$ is directly proportional to the internal photon density $S(t)$:

$$P_f(t) = S(t)\left[\frac{V_{act}\eta hc}{\Gamma \tau_p \lambda_0}\right] \tag{15}$$

5.3 Simulation Parameters

The model utilizes the geometric and physical parameters detailed in table 6.

Parameter	Symbol	Description	Unit	Value
Lasing wavelength	λ_0	Operating wavelength	cm	1.55×10^{-4}
Active region volume	V_{act}	Volume of the laser cavity	cm^3	1.5×10^{-10}
Optical confinement	Γ	Fraction of optical mode in active layer	-	0.3
Spontaneous emission	β	Coupling factor	-	1.0×10^{-4}
Photon lifetime	τ_p	Cavity photon decay time	ps	2.0
Gain coefficient	g_0	Differential gain	cm^3/s	2.5×10^{-6}
Transparency density	N_0	Carrier density at transparency	cm^{-3}	1.0×10^{18}
Carrier lifetime	τ_n	Electron-hole recombination time	ps	1000
Quantum efficiency	η	Differential quantum efficiency per facet	-	0.4
Equilibrium carriers	N_e	Equilibrium carrier density	cm^{-3}	0

Table 6: Simulation parameters for the Quantum-Well LASER model.

5.4 Building the QW Laser Subsystem

The physical rate equations present a significant unit-scaling challenge, as typical values span from picoseconds (10^{-12}) to particle densities (10^{23} in SI). To guarantee numerical stability, all inputs are converted to pure SI units (meters, seconds, kilograms) via a mask initialization script prior to block evaluation. This script, shown in listing 4, acts as the physics engine's pre-processor by converting mixed-unit parameters and calculating fixed constant multipliers for the Simulink integrators.

```
classdef rate_equations_init
    methods(Static)
        function MaskInitialization(maskInitContext)
            ws = maskInitContext.MaskWorkspace;

            % Retrieve raw parameters
            lambda0_cm = ws.get('lambda0'); Vact_cm3 = ws.get('Vact');
            Gamma      = ws.get('Gamma');   beta     = ws.get('beta');
            tau_p_ps   = ws.get('tau_p');   g0_cm3_s = ws.get('g0');
            N0_cm3     = ws.get('N0');      tau_n_ps = ws.get('tau_n');
            eta        = ws.get('eta');     Ne_cm3   = ws.get('Ne');

            % Convert ALL parameters to pure SI units
            lambda0 = lambda0_cm * 1e-2;
            Vact    = Vact_cm3 * 1e-6;
            tau_p   = tau_p_ps * 1e-12;
            tau_n   = tau_n_ps * 1e-12;
            g0      = g0_cm3_s * 1e-6;
            N0      = N0_cm3 * 1e6;
            Ne      = Ne_cm3 * 1e6;

            % Constants
            q = 1.60218e-19; h = 6.626e-34; c = 2.9979e8;
            eps_val = 3.4e-23 * 1e-6;

            % Calculate multipliers in pure SI
            G_I       = 1 / (q * Vact);
            G_Ne      = Ne / tau_n;
            G_N_decay = 1 / tau_n;
            G_S_decay = 1 / tau_p;
            G_Pf      = (Vact * eta * h * c) / (Gamma * tau_p * lambda0);

            % Push variables
            ws.set('G_I', G_I); ws.set('G_Ne', G_Ne);
            ws.set('G_N_decay', G_N_decay); ws.set('G_S_decay', G_S_decay);
            ws.set('G_Pf', G_Pf); ws.set('eps_val', eps_val);
            ws.set('g0', g0); ws.set('N0', N0);
        end
    end
end
```

Listing 4: Mask initialization code for the QW Laser subsystem managing pure SI unit conversion and calculating constant multipliers.

Unlike the small-signal models, this subsystem employs closed-loop time-domain numerical integration. Two `1/s` Integrator blocks are utilized to continually update $N(t)$ and $S(t)$. Mathematical operators explicitly recreate the polynomial $(N - N_0)(1 - \epsilon S)S$ gain-saturation mechanism at each time step, and the optical output P_f is extracted via the linear gain `G_Pf` applied to the photon density.

5.5 Top-Level Model and Test Harness

The main canvas features a comprehensive Test Harness utilizing a Multiport Switch. This structure allows seamless toggling between an unbiased (Case 1) or pre-biased (Case 2) current source to explore the physical extremes of the laser.

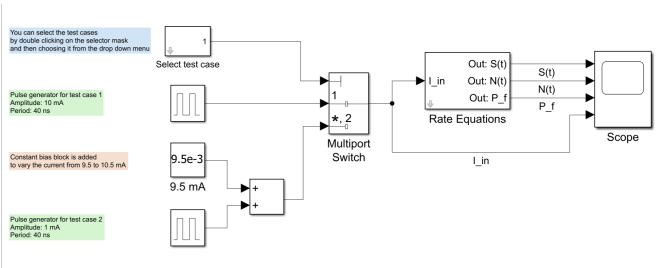
- **The Unipolar Variant (Pulse Generator):** This configuration provides an unbiased 10 mA pulse, or can be configured to act as an "Ideal Hot-Start" starting instantly at maximum current.
- **The Bipolar Variant (Signal Generator):** This variant uses a Signal Generator outputting a ± 0.5 mA square wave summed with a 10 mA DC bias. This starts the simulation low at 9.5 mA, realistically recreating the physical "Cold-Start" transient when powering up the device before modulation begins.

5.6 Solver Configuration

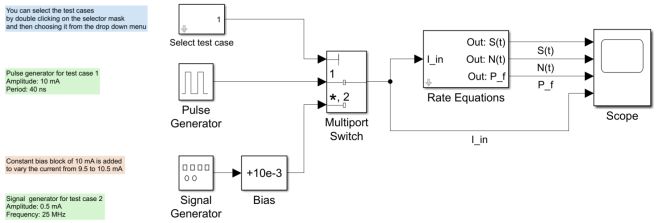
Due to the highly stiff nature of these coupled differential equations, the Simulink solver must be rigidly defined. The solver is set to **Fixed-step (ode4 Runge-Kutta)** with an extremely fine step size of `1e-13` seconds (100 fs) to maintain absolute numerical stability during the rapid photon bursts. Additionally, the gain multiplexers and arbitrary scaling blocks were avoided to feed raw, unscaled SI signals directly into a 4-panel scope layout.

5.7 Simulink Block Diagrams

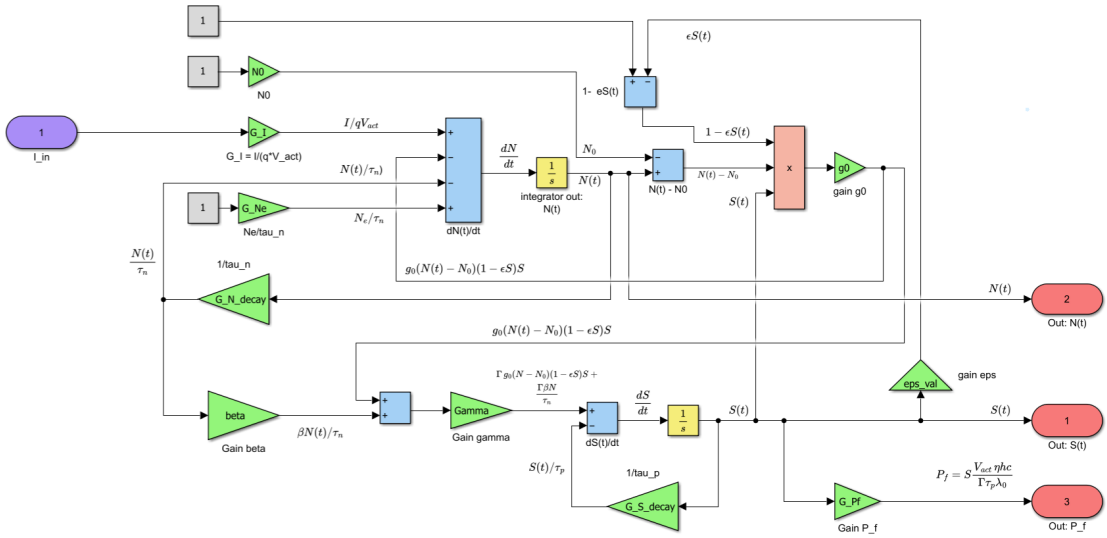
The dual test-harness arrangements and the internal rate-equation architecture are shown in fig. 5.



(a) Unipolar Pulse.



(b) Bipolar Signal.

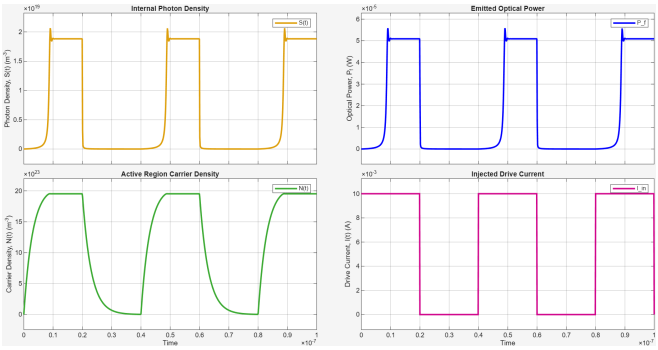


(c) Rate Equations.

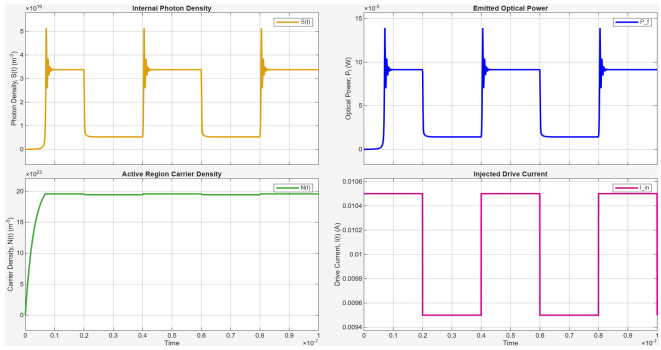
Figure 5: Simulink layouts for the QW Laser displaying the Hot-Start vs Cold-Start test benches, alongside the complex time-domain integration core.

5.8 Simulation Results and Analysis

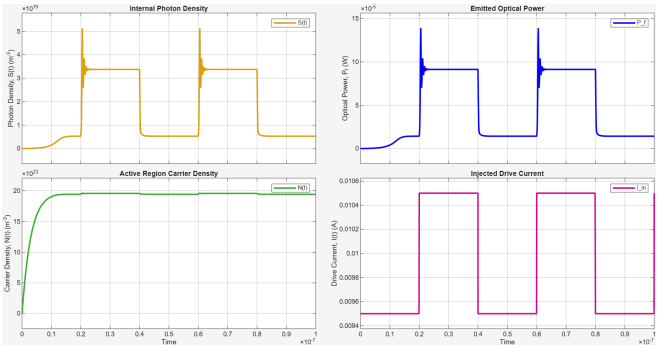
Using the robust multi-port harness, three distinct temporal behaviors were observed, as captured in fig. 6.



(a) Case 1: Unbiased.



(b) Case 2A: Hot-Start.



(c) Case 2B: Cold-Start.

Figure 6: Output profiles capturing turn-on delay (a), instantaneous clamped gain (b), and the realistic progression from empty cavity to steady-state modulation (c).

Case 1: Unbiased Pulse (0 to 10 mA). Grounded at 0 mA, the active region is completely depleted of electrons ($N \approx 0$). When the 10 mA pulse hits, electrons must first slowly fill the conduction band. This process provokes a massive ~ 0.4 ns turn-on delay before crossing the transparency threshold (N_0), after which population inversion triggers severe relaxation oscillations.

Case 2A: Ideal Hot-Start (10.5 mA clamping). Here, the simulation assumes the laser has been running continuously. The drive defaults immediately to 10.5 mA at $t = 0$. The carrier density is instantly clamped at its threshold boundary, avoiding any initialization transient entirely.

Case 2B: The Cold-Start Transient Timeline. The bipolar signal generator reveals the most accurate physical progression of a laser powering on from a dead state before transmitting data:

- **0 to 10 ns (Pumping):** The simulation starts with 9.5 mA injected into an empty cavity. Power and photon density remain perfectly flat at zero as electrons exponentially fill the conduction band.
- **10 to 15 ns (Turn-On & Ringing):** The carrier density eventually crosses N_0. The laser achieves inversion, violently turns on, and generates the initial burst of ringing.
- **15 to 20 ns (Steady-State Bias):** The ringing fully dampens. The horizontal power baseline is the physical manifestation of the 9.5 mA DC Bias. The laser is fully saturated and idling.
- **20 ns Onward (Modulation):** The data transmission finally begins, stepping the current from 9.5 mA to 10.5 mA. Because the laser spent the first 20 ns clamping its carrier density at the threshold, this modulation pulse experiences exactly zero turn-on delay.

6. Summary and Conclusions

This report successfully documented the step-by-section methodology for modeling three fundamental optical transmitter components within the MATLAB/Simulink environment. By utilizing subsystem encapsulation, mask initialization

scripts, and customized ODE solvers, we bridged the gap between theoretical semiconductor physics and practical, time-domain digital simulations.

A comparative summary of the modeled components is provided in table 7.

Component	Modelling Approach	Key Dynamic	System Implication
LED	Small-signal (1st-order)	Exponential Rise/Decay	Limited to low-speed links ($<$ 100 MHz) due to spontaneous emission lifetime (τ_r).
FP Laser	Small-signal (2nd-order)	Relaxation Oscillations	Capable of multi-GHz speeds, but prone to high-frequency ringing and intersymbol interference.
QW Laser	Large-signal (Non-linear)	Turn-on Delay & Clamping	Highly accurate for capturing physical delay from dead-starts. Pre-biasing completely eliminates the delay penalty.

Table 7: Comparative summary of the three modeled optical transmitters.

Key Takeaways:

- **Subsystem Encapsulation:** Utilizing Simulink Masks and `classdef` initialization scripts significantly reduces canvas clutter and guarantees that all calculations are performed in standardized SI units before runtime.
- **Solver Sensitivity:** Optical phenomena occur on nanosecond (carrier lifetime) and picosecond (photon lifetime) scales. Failing to rigidly clamp the maximum step-size of the ODE solvers (e.g., to 1 ps or 100 fs) causes variable-step algorithms to completely miss high-frequency relaxation oscillations.
- **Biasing is Critical:** The Quantum-Well laser model proved that attempting to drive a laser from an unbiased state (0 mA) introduces a massive turn-on penalty. Operating optical transmitters with a steady-state DC bias slightly above the lasing threshold is necessary for high-speed digital modulation.

References

[1] C. F. de Melo, C. A. Lima, L. D. de Alcantara, R. O. dos Santos, and J. C. W. A. Costa, "Simulink toolbox for simulation and analysis of optical fiber links," *Proceedings of SPIE*, vol. 3831, 1999, base reference paper for all model parameters and governing equations. [Online]. Available: https://www.spiedigitallibrary.org/conference-proceedings-of-spie

[2] MathWorks, "Create block masks," 2024. [Online]. Available: https://www.mathworks.com/help/simulink/block-masks.html

[3] MathWorks, Inc., "Transfer fcn block," 2024. [Online]. Available: https://www.mathworks.com/help/simulink/slref/transferfcn.html

[4] MathWorks Documentation, "Choose a solver," 2024. [Online]. Available: https://www.mathworks.com/help/simulink/ug/choose-a-solver.html